EMBODIED SEMANTIC ASSET RECALL ROBOT

A PROJECT REPORT PHASE– I

submitted by

JIBIN C BASIL (MGP23RA049)

NIDHI N (MGP23RA060)

SIDHARTH SAJITH (MGP23RA072)

THARUN OOMMEN JACOB (MGP23RA075)

in partial fulfilment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY



DEPARTMENT OF ELECTRONICS ENGINEERING

SAINTGITS COLLEGE OF ENGINEERING (AUTONOMOUS)

KOTTAYAM

OCTOBER 2026

DEPARTMENT OF ELECTRONICS ENGINEERING

SAINTGITS COLLEGE OF ENGINEERING

(AUTONOMOUS),KOTTAYAM



CERTIFICATE

Certified that this report entitled **Embodied Semantic Asset Recall Robot** is submitted by **Jibin C Basil(MGP23RA049)**, **Nidhi N(MGP23RA060)**, **Sidharth Sajith(MGP23RA072)**, **Tharun Oommen Jacob(MGP23RA075)** during the academic year **2026–27** in partial fulfillment of the requirements for the award of the Degree of Bachelor of Technology in **Robotics and Automation** of Saintgits College of Engineering (Autonomous) under APJ Abdul Kalam Technological University.

Er. Chinn Mohanan
Project Guide
Assistant Professor
Dept. of Electronics Engineering

Er. Rizwana Akbar
Project Co-ordinator
Assistant Professor
Dept. of Electronics Engineering

Dr. Starlet Ben Alex
Program Co-ordinator
Associate Professor
Dept. of Electronics Engineering

Place: Kottayam

Date: _____

ACKNOWLEDGEMENTS

We express our profound gratitude to all those who have supported us. Our sincere thanks to:

Er. Chinn Mohanan, under whose inspiring guidance and supervision this research was carried out. His insistent attitude and timely help led to the successful completion of this project.

Er. Rizwana Akbar, for providing timely instructions and fruitful suggestions as the Project Co-ordinator.

Dr. Lakshmi M Hari, Chairman of the panel, for her support and motivation.

Dr. Shiny G, Head, Department of Electronics Engineering for providing all the departmental facilities.

Dr. Starlet Ben Alex, Program Coordinator (ECE), Department of Electronics Engineering for the kind support and motivation.

Dr. Sudha T, Principal, Saintgits College of Engineering (Autonomous) for providing an excellent ambiance that laid potentially strong foundation for our work.

All the staff and students of the Department of Electronics Engineering for rendering help and support.

Our parents, friends and well-wishers for their support and timely help.

Above all we thank the Almighty God who made all this possible for us by being there with us always.

ABSTRACT

KEYWORDS: Semantic asset recall; Object search; Bayesian filtering; Open-vocabulary perception; Edge robotics; Offline localisation.

Shared laboratories, workshops and maintenance rooms routinely lose track of handheld tools and instruments: manual logs demand discipline that small teams do not maintain, RFID tagging needs per-item infrastructure, and commercial inventory robots cost far beyond a student budget and assume cloud connectivity. This project, **Embodied Semantic Asset Recall Robot**, finds a named object inside a single small room by building a persistent semantic memory of the space and reasoning probabilistically about where the object is most likely to be.

The robot uses an ordinary smartphone as its sensor — ARCore visual-inertial odometry for pose, the camera for RGB frames and the IMU for motion events — and a local PC as its intelligence: a WebSocket bridge streams pose and image data into a semantic scene graph, and a Bayesian search core ranks the most probable location and plans the next best view. The system is fully offline by design: no cloud service is used at any stage.

Phase I delivered the phone-to-bridge-to-PC pipeline (verified end to end by host loopback), the unit-tested Bayesian search core (simulation entropy 11.83 to 9.39 bits over 20 steps), the sensing application compiled to APK v1, the SolidWorks mechanical concept and BOM v0.1 (Rs. 7,390 planned). Nothing has yet been demonstrated on hardware; Phase II covers the mobile base, ROS 2 integration and the on-robot demonstration. This report states explicitly what has and has not been achieved.

Contents

ABSTRACT **4**

LIST OF TABLES **7**

LIST OF FIGURES **8**

1 INTRODUCTION **10**

1.1 Background and Motivation 10

1.2 Existing Systems and Approaches 11

1.3 Problem Statement 12

1.4 Scope and Constraints 12

1.5 Project Evolution 13

1.6 Report Organisation 14

2 LITERATURE REVIEW **15**

2.1 Open-Vocabulary Semantic Mapping 15

2.2 Semantic Scene Graphs for Manipulation 16

2.3 Object-Goal Navigation and Search 16

2.4 Efficient Edge Perception 17

2.5 Supporting Frameworks 18

2.6 Research Gap 18

3 OBJECTIVES **20**

3.1 Overall Aim 20

3.2 Project Objectives 20

3.3 Phase-I Scope 21

3.4 Deliverables 22

4 PROJECT PLAN AND TASK ALLOCATION **23**

4.1 Phase-I Plan 23

4.2 Task Allocation 24

4.3 Phase-II Plan 25

5 DESIGN AND METHODOLOGY **26**

5.1 System Architecture 26

5.2 Mechanical Design 28

5.3 Software Design 29

5.3.1 Phone Application 29

5.3.2 Communication Bridge 29

5.3.3 Search Core 30

5.4 Development Methodology 30

5.5 Phase-I Implementation Status 31

6 PRELIMINARY RESULTS **33**

6.1 Bayesian Search Simulation 33

6.2 Communication Bridge Verification 34

6.3 IMU Streaming Prototype 34

6.4 Mechanical Design Output 34

6.5 What Has Not Yet Been Demonstrated 35

7 CONCLUSION **36**

List of Tables

1.1 Comparison of existing asset-finding approaches 12

2.1 Surveyed systems against the project requirements 19

4.1 Project Plan for Phase I 23

4.2 Task allocation matrix (X = responsible track) 24

5.1 Phase-I implementation status by component 32

List of Figures

1.1 Superseded early concept mock of a 4WD base (not the current design) . 13

5.1 System block diagram (phone sensing, local PC intelligence, ESP32 actuation) 26
5.2 Example semantic-scene-graph record for one tracked asset (illustrative schema) 27
5.3 Complete 2WD mobile-base assembly concept (SolidWorks; not fabricated) 28
5.4 Sensing phone seated in the tilt holder 28
5.5 Next-best-view search loop flowchart 31

6.1 Bayesian search simulation: belief heatmaps and entropy decay over 20 steps 33
6.2 BOM cost evolution: draft architecture versus revised bench-rig and base plan prices (Rs.7,390 total) 35

ABBREVIATIONS

APK	Android Package Kit
ARCore	Google's augmented-reality platform for visual-inertial tracking
AWQ	Activation-aware Weight Quantization
BOM	Bill of Materials
BT	Behaviour Tree
CAD	Computer-Aided Design
DoF	Degrees of Freedom
EKF	Extended Kalman Filter
FOV	Field of View
IMU	Inertial Measurement Unit
JPEG	Joint Photographic Experts Group
JSON	JavaScript Object Notation
mAP	mean Average Precision
MPPI	Model Predictive Path Integral (controller)
NBV	Next-Best-View
ROS	Robot Operating System
ToF	Time of Flight (distance sensor)
VIO	Visual-Inertial Odometry
VLM	Vision-Language Model
WS	WebSocket

CHAPTER 1

INTRODUCTION

This chapter introduces the project. **Phase I** in this report means the B.Tech Phase-I report period (June–October 2026). The project — an **embodied semantic asset recall robot** — is a low-cost mobile platform that builds a semantic memory of an indoor laboratory scene and recalls missing objects on request, fully offline.

1.1 Background and Motivation

Shared laboratories, maintenance rooms and small workshops routinely lose track of hand tools and instruments. A multimeter, a soldering station or a set of calipers is borrowed, carried to a bench, and left there; the next user spends valuable time searching for it. The domain of this project is therefore **industrial asset tracking** and laboratory management, restricted to a single indoor room of about 5×5 m containing some 10–15 everyday lab object classes.

Closed-circuit television (CCTV) only partly helps: it records, but it cannot answer the question "where is the multimeter now?" without a human scrubbing through hours of footage. Manual stock registers and sign-out logs are slow, depend on tagging discipline, and fall out of date within days. Commercial inventory robots of the Simbe Tally class address larger retail floors but are expensive, depend on cloud services, and are not designed for a college laboratory that must work offline on a student budget. The motivation for this project is a system that accepts a natural-language query on an ordinary phone ("find the red multimeter") and answers it from its own memory of the room, without per-item tags, without cloud connectivity, and at a cost a student team can afford.

1.2 Existing Systems and Approaches

Four established approaches to finding assets in indoor spaces were studied before fixing the project direction. Academic work on semantic navigation robots, notably zero-shot active visual search [11] and semantic reasoning for object-goal navigation [3], is reviewed in detail in Chapter 2; the summary here concentrates on what each existing approach costs and what kind of query it can answer.

Manual logs and sign-out registers are the default in most college labs. They cost nothing, but every entry depends on human discipline, the record describes where an item should be rather than where it is, and searching means walking the room. There is no query capability beyond reading the book.

RFID tagging attaches a radio tag to each item and places readers at doors or shelves. It answers "which tagged items passed this reader" well, but every item must be tagged and maintained, readers and wiring must be installed at each choke point, and an untagged or damaged tag is invisible to the system. The per-item infrastructure cost and the tagging discipline it demands make it unattractive for a small teaching lab.

CCTV with human review records continuously but has no semantic understanding of the scene. Finding an object means a person replays footage and recognises the item by eye. It cannot accept a query such as "show me the oscilloscope probes" and it raises storage and privacy overheads that a lab does not need.

Commercial inventory robots (the Simbe Tally class of shelf-scanning platforms) automate the walk-around, but they are priced for retail chains, assume large, well-structured aisles, and generally depend on vendor cloud services for perception. They are neither low-cost nor offline, and they scan bar-coded shelves rather than recalling arbitrary lab tools from memory.

Academic semantic-navigation robots form the fifth and most relevant line of work. Systems such as ZAVIS [11] and SEEK [3] drive a robot toward an object described in language, using learned visual semantics rather than tags. Their limitation, analysed in Chapter 2, is that they assume substantial on-board or cloud compute and target large spaces; none offers low-cost, offline, single-room semantic recall with an ordinary phone as the sensor. That gap is exactly what this project is designed to fill.

Table 1.1 compares the approaches on the criteria that matter for this project.

Table 1.1: Comparison of existing asset-finding approaches

Approach	Cost	Query capability	Offline operation
Manual logs	Negligible	None (read the register)	Yes, but no localisation
RFID tagging	Medium–high per item	Tag identity at readers	Yes, with installed readers
CCTV + human review	Medium	None (manual replay)	Yes, but needs a reviewer
Commercial inventory robots	High	Shelf/bar-code scans	Usually cloud-dependent
Semantic-navigation research	High compute	Natural-language goals	Rarely; often needs cloud

The conclusion of the comparison is that no existing approach provides natural-language recall of untagged lab objects, offline, at student-project cost. This project targets that empty cell of the table.

1.3 Problem Statement

Design, in B.Tech Phase I, the architecture, mechanical concept, search intelligence and communication layer of a low-cost offline robot that, for a single 5×5 m laboratory room containing 10–15 common object classes, accepts a natural-language object query entered on a second phone and returns the remembered location of the object, verified against its stored semantic map. The Phase-I work covers design, component-level implementation and feasibility evidence; integration of the full recall loop on hardware is explicitly deferred to Phase II.

1.4 Scope and Constraints

The work is bounded by four students, one semester, and a semester-scale budget. In scope for Phase I are: the system architecture; the mechanical concept design of the mobile base; a simulation-validated Bayesian planner (unit-tested; not yet run on hardware); the phone streaming application and its WebSocket bridge (bridge host-loopback PASS; ROS adapter code-complete but not executed, since no ROS 2 environment was

available during Phase I); a literature survey fixing the perception roadmap; and a bill of materials held under the budget target. Out of scope for Phase I, and hence for this report's claims, are multi-room operation, manipulation of objects, human-aware navigation, on-hardware autonomy, and any cloud dependency: the design is offline by requirement.

1.5 Project Evolution

The problem statement has been stable since June 2026, but the delivery order changed as evidence accumulated. Figure 1.1 shows the earliest concept: a four-wheel-drive base with an on-board computer, sketched before the zeroth review. That sketch is **superseded** and is included only as a record of the starting point.

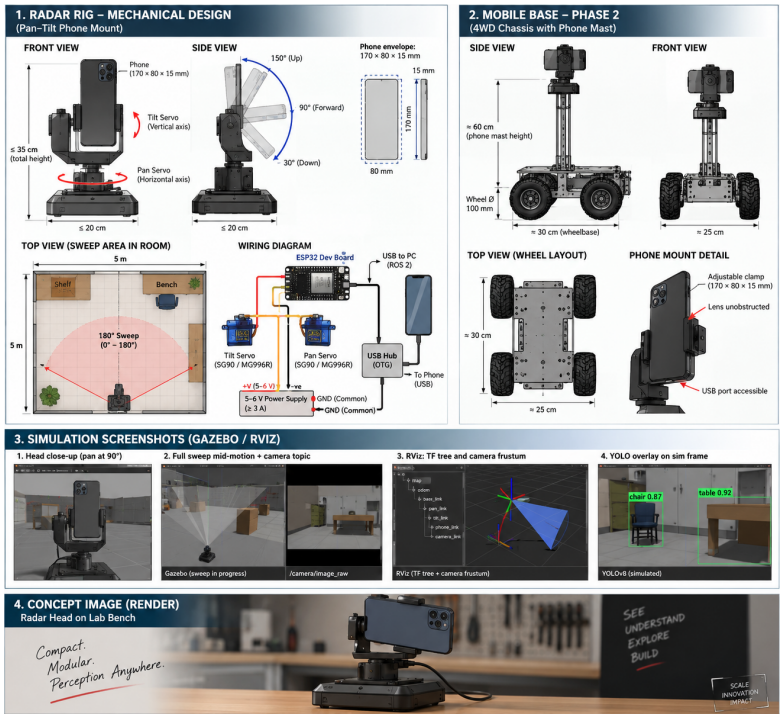


Figure 1.1: Superseded early concept mock of a 4WD base (not the current design)

The dated pivot summary is as follows. In July 2026 the zeroth review presented a Jetson-centric 4WD mobile base with voice input. On 8 September 2026 the team, with guide approval, pivoted to a hybrid plan: a static pan-tilt bench rig (milestone M2a) first for software bring-up, with the mobile base deferred to Phase II. On 2 October 2026 the mobile base itself was downsized to a 2WD differential-drive concept with a phone-tilt head, dropping the Jetson, the 4WD transmission and the oversized motor drivers from

the bill of materials. The V1 Final Internal Review of 6 October 2026 presented this revised architecture. Chapter 4 gives the full dated chronology; Chapter 5 describes the current design.

1.6 Report Organisation

The remainder of this report is organised as follows:

- Chapter 2 reviews the literature on open-vocabulary semantic mapping, semantic scene graphs, object-goal navigation and search, and efficient edge perception, and closes with the research gap this project addresses.
- Chapter 3 states the project objectives as locked at the final internal review of 6 October 2026, and marks which of them received design-level or component-level treatment in Phase I.
- Chapter 4 records the project plan: the task allocation across the team, the dated chronology of design decisions, and the Phase-II roadmap.
- Chapter 5 describes the design and methodology: the system architecture, the mechanical concept, the software design and the Bayesian search policy.
- Chapter 6 presents the Phase-I results that do exist — the bridge verification, the search-engine tests and the simulation study — together with their verification status, and states explicitly what has not yet been demonstrated.
- Chapter 7 concludes the report and lists the limitations carried into Phase II.

CHAPTER 2

LITERATURE REVIEW

The literature is organised thematically rather than as disconnected paper summaries: open-vocabulary semantic mapping, semantic scene graphs, object-goal navigation and search, efficient edge perception, and supporting frameworks. Each work is noted with its contribution, method and limitation, and the chapter closes with the research gap that motivates this project. Every substantive claim below is cited; all cited entries were verified against publisher or arXiv metadata during report preparation.

2.1 Open-Vocabulary Semantic Mapping

The core problem of remembering *what is where* in language terms was opened by three closely related systems. **CLIP-Fields** learns a neural implicit map that attaches CLIP language embeddings to 3D locations from posed RGB images, so that a query such as "red mug" resolves to map coordinates without retraining for new classes [12]. Its method is weak supervision: back-project detections into space and distil them into a continuous field. Its limitation for this project is memory and speed: implicit fields are heavy to query at frame rate on weak hardware, and the scenes are assumed static during mapping.

ConceptFusion extends the idea to open-set multimodal 3D mapping by fusing pixel-aligned vision-language features with geometric mapping, producing a dense map that answers arbitrary language, image or audio queries [7]. The method couples a foundation-model feature extractor with a surfel or voxel map. The limitation is compute: the feature extraction it relies on runs far below real time on edge hardware and in practice assumes offline or workstation-class processing.

VLMaps precomputes a grid map in which each cell carries a fused vision-language embedding, enabling language-indexed navigation goals for a robot [6]. Its strength is the direct bridge from map query to navigation target. Its limitation is again scale and dynamism: building and refreshing the embedding grid is expensive, and moved objects

invalidate stale cells, which matters in a laboratory where tools move daily.

Together these three fix the representation this project needs (language features grounded in space) while showing that none of them runs natively on a cheap offline platform.

2.2 Semantic Scene Graphs for Manipulation

Where metric maps remember positions, **scene graphs** remember structure: which objects exist and how they relate. **ConceptGraphs** builds open-vocabulary 3D scene graphs of objects with language captions, usable for both perception and task planning [4]. Its method segments objects in 3D, captions each with a vision-language model, and links them into a graph. Its limitation is the two-stage cost: segmentation plus per-object captioning is computationally heavy, and graph maintenance over long deployments is not addressed.

DovSG (dynamic open-vocabulary scene graphs) tackles exactly that long-term dynamic case, updating the graph as objects move and supporting language-guided mobile manipulation over extended periods [14]. Its method adds change detection and incremental graph updates. Its limitation is that continuous dynamic updating is the most expensive regime of all: it assumes sustained perception compute that a phone-plus-local-PC setup cannot yet guarantee, and its demonstrations target manipulation platforms well above student-project cost.

The lesson for this project is that a lightweight, slowly updated scene memory — rather than a continuously maintained dynamic graph — is the realistic Phase-II target for a single room.

2.3 Object-Goal Navigation and Search

Given a memory or a live view, the robot must still *find* the object. **ZAVIS** formulates zero-shot active visual search: an agent that looks around intelligently to locate a described object, without task-specific training [11]. Its method combines open-vocabulary detection with a view-selection policy. Its limitation is environment scale and the sim-to-real gap: policies tuned in simulation degrade with real sensor noise, and the work targets assistant robots rather than minimal student hardware.

SEEK adds semantic reasoning for object-goal navigation in real inspection tasks,

using structured scene understanding to guide search in industrial settings [3]. Its method couples semantic maps with frontier-style exploration. Its limitation is that it assumes a moving base with reliable localisation from the start — precisely the capability Phase I defers — and its compute budget exceeds what an offline phone-based pipeline offers today.

L3MVN brings large language models into visual target navigation, using LLM commonsense (e.g. kitchens contain microwaves) to propose where to look next [15]. Its method prompts an LLM for likely object locations given the scene context. Its limitation is dependence on large models and, typically, connectivity: querying an LLM at decision time conflicts with the offline constraint, and the base-movement requirement remains.

These three motivate the Phase-I choice of a **probabilistic search core** (Chapter 5) that needs no learned policy and no connectivity: a Bayesian belief grid with a next-best-view rule captures the "where to look next" problem in its cheapest rigorous form.

2.4 Efficient Edge Perception

All of the above assumes a detector that can recognise arbitrary lab objects on weak hardware. **YOLO-World** provides real-time open-vocabulary object detection by aligning a YOLO backbone with language embeddings, so new classes are added through text prompts rather than retraining [1]. Its method is a vision-language neck on a fast detector; its limitation is the familiar accuracy–speed trade-off and reduced robustness on small or cluttered objects of exactly the kind found on lab benches.

Florence-2 takes the opposite end of the trade-off: a unified foundation model covering detection, grounding, captioning and segmentation through prompt-based task specification [13]. Its method is large-scale multi-task pretraining; its limitation is size, making it a workstation-side or distilled-only option for this project rather than a phone-side detector.

Grounding DINO occupies the accuracy end of the open-set spectrum: it marries the DINO detector family with grounded language pre-training, so arbitrary concepts can be detected from text prompts without class-specific training [9]. Its method is a text-conditioned Transformer detector; its limitation is latency and compute cost, which places it as the third candidate in the Phase-II benchmark rather than a real-time front

end.

AWQ (activation-aware weight quantisation) is the bridge between these worlds: it compresses large models for edge deployment with small accuracy loss by protecting the few salient weights [8]. Its method profiles activations to guide low-bit quantisation. For this project it is a Phase-II direction, not a Phase-I result: it shows how a Florence-class model could eventually be served from the local PC or a future edge board without cloud access.

The perception roadmap for Phase II is therefore a three-way benchmark of YOLO-World, Florence-2 and Grounding DINO on the project's own 10-class lab set, measured on mAP, FPS and RAM on the local PC, with AWQ-class quantisation as the deployment path.

2.5 Supporting Frameworks

Three mature frameworks underpin the navigation stack. **OctoMap** provides an efficient probabilistic 3D occupancy representation based on octrees, the standard substrate for mapping with noisy range data [5]. **Behaviour trees**, as systematised by Colledanchise and Ögren, provide the modular mission structure (recall, verify, search) that will orchestrate the robot's tasks [2]. **EKF localisation** via the `robot_localization` package fuses odometry, IMU and visual odometry into a continuous pose estimate [10]. None of the three was brought up in Phase I; all are Phase-II integration items, cited here because the architecture (Chapter 5) reserves their interfaces.

2.6 Research Gap

Table 2.1 summarises how the surveyed systems score against the five requirements of this project: low cost, offline operation, single-room focus, phone-as-sensor compatibility, and probabilistic search under uncertainty.

Table 2.1: Surveyed systems against the project requirements

System	Low cost	Offline	Phone sensor
CLIP-Fields [12]	No	Partly	No
ConceptFusion [7]	No	Workstation	No
VLMaps [6]	No	Workstation	No
ConceptGraphs [4]	No	Workstation	No
DovSG [14]	No	No	No
ZAVIS [11]	No	Partly	No
SEEK [3]	Platform-dependent	Partly	No
L3MVN [15]	No (LLM)	No	No
YOLO-World [1]	Yes	Yes	Feasible
Florence-2 [13]	No	Workstation	No
Grounding DINO [9]	No	Workstation	No

The synthesis is direct. Existing systems either need cloud or workstation-class compute, or navigate large spaces with a moving base they assume already works, or recognise only pre-tagged items. None provides low-cost, offline, single-room semantic recall with a phone as the sensor and an explicit probabilistic object-search policy. The closest perception match, YOLO-World, has no memory or search layer of its own. This gap — semantic recall that is simultaneously cheap, offline, phone-sensed and uncertainty-aware — is the space this project is designed to occupy, and every Phase-I component in Chapters 5 and 6 is traceable to one row of this table.

CHAPTER 3

OBJECTIVES

3.1 Overall Aim

The overall aim of the project is an offline, low-cost robot that maintains a semantic memory of a single laboratory room and recalls the location of a requested tool or instrument from a natural-language query entered on an ordinary phone. Phase I of the B.Tech programme (June–October 2026) covers the design, component-level implementation and feasibility evidence for that system; Phase II covers hardware integration, on-robot autonomy and the end-semester demonstration.

3.2 Project Objectives

The project objectives below are the formulation locked at the final internal review of 6 October 2026. Each is stated with the concrete target the team committed to:

1. Build a reliable 2-wheel-drive differential-drive mobile base with fused localisation (ARCore visual-inertial odometry, optional wheel odometry and an extended Kalman filter) achieving a pose accuracy of 0.1 m or better in a 5×5 m room [10].

2. Develop a two-stage perception pipeline on the local PC: continuous open-vocabulary detection with YOLO-World [1] plus event-triggered vision-language verification with Florence-2 [13] at keyframes, benchmarked against Grounding DINO [9] on the project's RTX 4060 machine.

3. Implement a persistent semantic scene graph, JSON-backed, storing for each asset a unique identifier, class, 3-D position, timestamp, confidence, spatial relationships (on, inside, near) and carrier associations, surviving power cycles.

4. Enable natural-language asset recall through the interaction phone application: the query is parsed for intent, the robot navigates to the last-known location,

verifies the object visually, and reports the result with a bounding box and on-screen confirmation.

5. Implement Bayesian active search for moved or missing objects: a probability distribution over discrete candidate zones, updated from history, semantic and carrier priors and from detection-failure-aware updates, shipped as the unit-tested `soma_brain` module.

6. Demonstrate three scenarios in a real laboratory with the internet disabled: stationary tool recall, moved-object search, and a safety compliance inspection.

7. Quantify the system against four baselines (random, nearest-zone, last-known-only and semantic-prior) on success rate, time-to-find, path length, viewpoints visited and runtime (RAM, FPS, latency).

8. Provide a two-phone, fully offline human–robot interface: the on-robot sensing phone (`soma-app`: visual-inertial odometry and stream) and the user's interaction phone (queries, memory cards, heatmap), with an ultrasonic emergency stop for close-range safety.

3.3 Phase-I Scope

Not all eight objectives can be completed inside Phase I, and this report marks precisely where each stands at the report date. The Phase-I treatment of the objectives is as follows:

- Objective 1: mechanical concept design in CAD with drive and tilt sizing calculations (the 0.1 m accuracy target itself is untested — no localisation has run on hardware).

- Objective 2: the benchmark is scoped and the candidates selected; no benchmark has been run yet.

- Objective 3: the scene-graph schema is designed; no persistent graph has been populated from real perception.

- Objective 4: the query-to-verification flow is designed; the interaction application is not built (the Phase-I app is the sensing-side application only).

- Objective 5: fully addressed at component level — the `soma_brain` module is implemented and its invariants unit-tested in simulation.
- Objective 6: not addressed (requires the real laboratory demonstration in Phase II).
- Objective 7: not addressed (requires an integrated system and recorded trials).
- Objective 8: partially addressed — the sensing-side phone application exists as APK v1 and the WebSocket bridge is host-verified; the interaction application, the ultrasonic emergency stop and the integrated two-phone demonstration remain for Phase II.

3.4 Deliverables

Phase I produced the following deliverables, all of which exist in the project repositories at the report date: the SolidWorks CAD assembly of the 2WD base with renders; the `soma_brain` Bayesian search package with its unit tests; the `bot_bridge` WebSocket bridge with its unit and end-to-end tests; the phone application compiled to APK v1 in August 2026 (host-side JUnit tests exist but were not re-run during report preparation); the `soma_description` ROS 2 description package (URDF, display launch and RViz configuration); BOM v0.1 for the bench rig and the mobile base; and this report.

CHAPTER 4

PROJECT PLAN AND TASK ALLOCATION

This chapter records how Phase I was actually executed, who did what, and what follows. “Phase I” throughout means the B.Tech Phase-I period (June–October 2026); “M2a” means the bench-rig milestone, a separate hardware fixture described in Section 4.3.

4.1 Phase-I Plan

Table 4.1 lists the Phase-I work week by week as it happened, backed by the dated project chronology (chat records, review documents, commit history and release artefacts). Status is given as Done or In progress at the report date.

Table 4.1: Project Plan for Phase I

Week / period	Planned	Status
Jun W3–W4	Problem framing and team formation; architecture blueprint	Done
Jul	Zeroth review preparation and submission; IMU streaming prototype	Done
Aug	Phone app refactor and APK v1; repository hygiene; security audit	Done
Sep W1	Current project repository initialisation	Done
Sep W2	Hybrid-plan decision; M2a documentation (BOM, block diagram, micro-ROS and compute notes, week plan)	Done
Sep W3–W4	First review preparation and presentation	Done
Oct W1	CAD assembly and renders; URDF description; Bayesian grid and NBV planner; 2WD revision of BOM	Done
Oct W2	V1 Final Internal Review (2026-10-06); motor and battery shortlist; open-questions register	Done
Oct W3	Phase-I report writing and compilation	In progress

The June blueprint fixed the problem and the four-person team. July produced the zeroth-review report and deck (Jetson/4WD era) alongside an IMU streaming prototype that demonstrated the phone-as-sensor idea early. August produced the multi-screen rewrite of the phone application, compiled to APK v1, plus a security audit of the phone–bridge path. September opened the current repository, recorded the guide-approved hybrid decision (bench rig first, mobile base deferred), and generated the M2a planning documents. October added the CAD assembly with renders, the URDF description, the Bayesian search package with its planner, the downsized 2WD bill of materials, and the V1 Final Internal Review of 6 October 2026 with its motor/battery shortlist and open-questions register. Report writing in the third week of October closes the phase.

4.2 Task Allocation

Work is allocated by track across the four members. Table 4.2 shows the current allocation: Sidharth (software architecture and integration lead), Nidhi (perception and literature), Tharun (hardware, CAD and BOM lead) and Jibin (devices, testing and documentation).

Table 4.2: Task allocation matrix (X = responsible track)

Track / task	Sidharth	Nidhi	Tharun	Jibin
Software architecture and integration	X			
Perception and AI research		X		X
Bayesian search engine	X	X		
Phone app and bridge	X			X
Mechanical CAD and sizing			X	
BOM and procurement			X	X
Reviews and documentation	X	X	X	X

Note: Allocation reflects the team composition since September 2026. Work before September 2026 was carried out under an earlier team composition (the hardware lead changed in September 2026); June–August entries in Table 4.1 are therefore attributed by track (Software/AI versus Hardware/Integration) rather than by individual.

Software architecture, the Bayesian engine and the bridge sit with the Software/AI

track; mechanical design, sizing and procurement sit with the Hardware/Integration track; perception research and testing are shared; and all four members contribute to reviews and documentation.

4.3 Phase-II Plan

Phase II runs from the bench rig to the end-semester demonstration, which has a locked November 2026 deadline. Milestones:

- **M2b — mobile-base integration:** chassis build, wheel odometry, EKF fusion [10], OctoMap layer [5] and Nav2 bring-up.
- **M3 — real phone pipeline:** live bridge-to-ROS path, ESP32 micro-ROS head control, recorded rosbags and EKF tuning.
- **M4 — perception benchmark:** YOLO-World [1], Florence-2 [13] and Grounding DINO [9] on the 10-class lab set; detector selection and quantisation [8].
- **M5 — intelligence:** behaviour-tree recall, verify and search missions [2] over the Bayesian search core.
- **M6 — dashboard:** operator interface for queries and map inspection.
- **M7 — end-semester package:** integrated demonstration and documentation for the November 2026 review.

Two blockers are carried into Phase II and recorded in the open-questions register: final drive-motor selection and the battery architecture. Bench-rig parts are listed as to order in BOM v0.1; ordering was not confirmed at the report date.

CHAPTER 5

DESIGN AND METHODOLOGY

This is the longest chapter of the report. It describes the system architecture, the mechanical concept design, the software design and the development methodology used in Phase I, and it closes with an honest component-by-component implementation status. Throughout, "Phase I" means the B.Tech Phase-I period (June–October 2026) and "M2a" means the bench-rig milestone: a separate, not-yet-ordered static pan-tilt fixture planned as the first software bring-up vehicle in Phase II.

5.1 System Architecture

Figure 5.1 shows the system block diagram presented at the V1 review. The design is organised in three tiers.

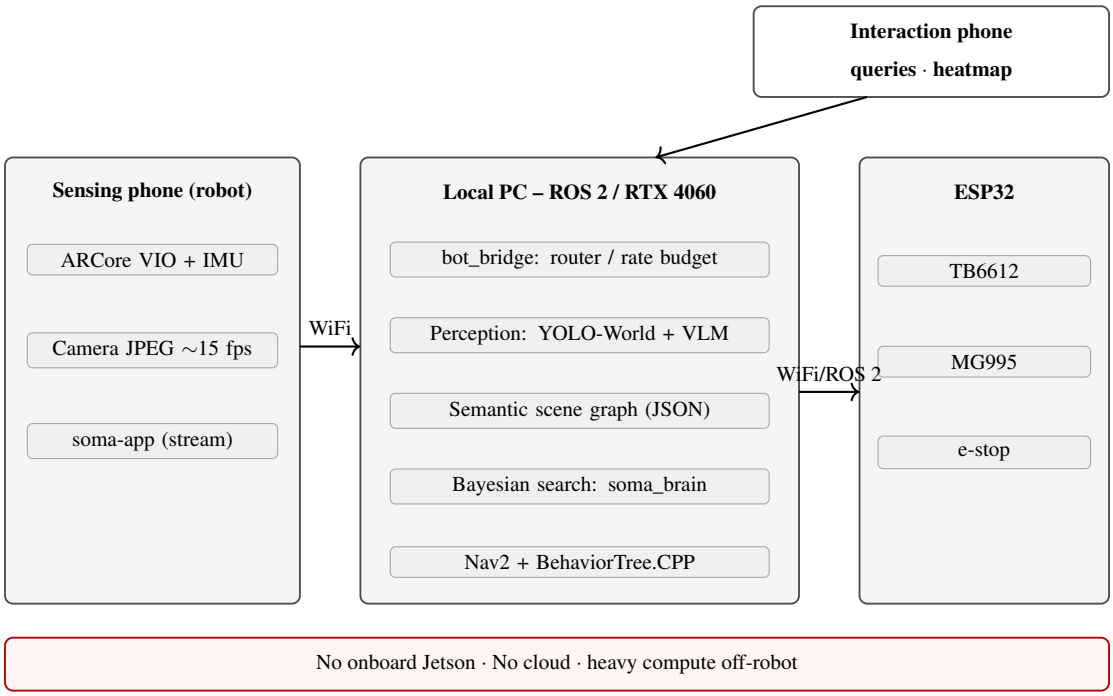


Figure 5.1: System block diagram (phone sensing, local PC intelligence, ESP32 actuation)

The **sensing phone** is carried on the robot's tilt head and provides ARCore visual-inertial odometry, Camera2 image frames and IMU streams at about 30 Hz for pose/IMU and about 15 fps for JPEG frames. The **local PC** (hosts the ROS 2 graph, the WebSocket bridge, perception, the scene memory, the Bayesian search and the behaviour-tree missions) performs all heavy computation over a WiFi-direct WebSocket link using a versioned protocol: a 22-byte binary frame header for image data plus JSON control messages (`protocol_v1`). The **ESP32 head controller** (planned, micro-ROS) will drive the pan-tilt servos and relay low-level sensor traffic.

Phase I implemented the phone, bridge and PC path end to end, together with the search core; the ESP32/micro-ROS tier and all mobility functions belong to Phase II. The compute-placement rationale is deliberately hybrid: no Jetson on the robot and no cloud anywhere in the demo path, satisfying the offline and cost constraints simultaneously.

The Jetson-centric 4WD concept of July 2026 was abandoned for exactly this reason. Quantisation of large perception models for edge deployment [8] is the recorded future direction for keeping that constraint as models grow.

The semantic scene graph ties the tiers together: every tracked asset carries a persistent record of the form illustrated in Figure 5.2, which survives power cycles and is updated as detections arrive.

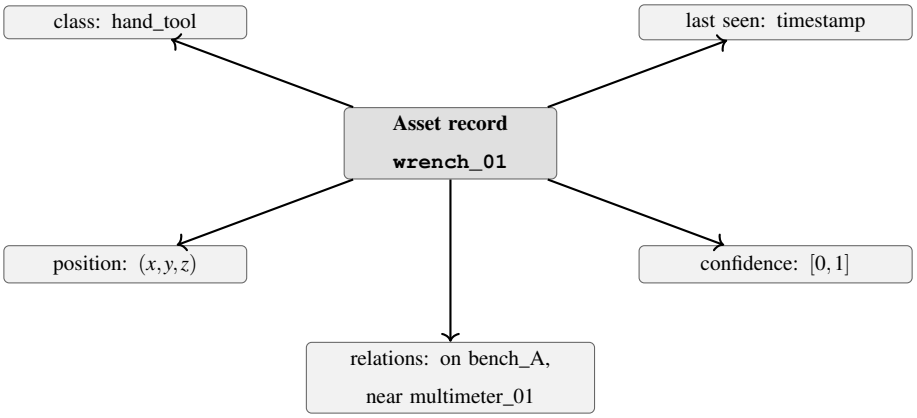


Figure 5.2: Example semantic-scene-graph record for one tracked asset (illustrative schema)

5.2 Mechanical Design

Figures 5.3 and 5.4 show the SolidWorks concept renders of the 2WD mobile base. The design is a hexagonal two-level platform: the lower level carries the differential drive (two driven 65 mm wheels with a front castor), battery and motor drivers; the upper component deck carries the compute-adjacent electronics and the phone-tilt head. The head is a single-axis tilt mount for the sensing phone driven by an MG995 servo, converting driving into a repeatable camera-sweep motion.

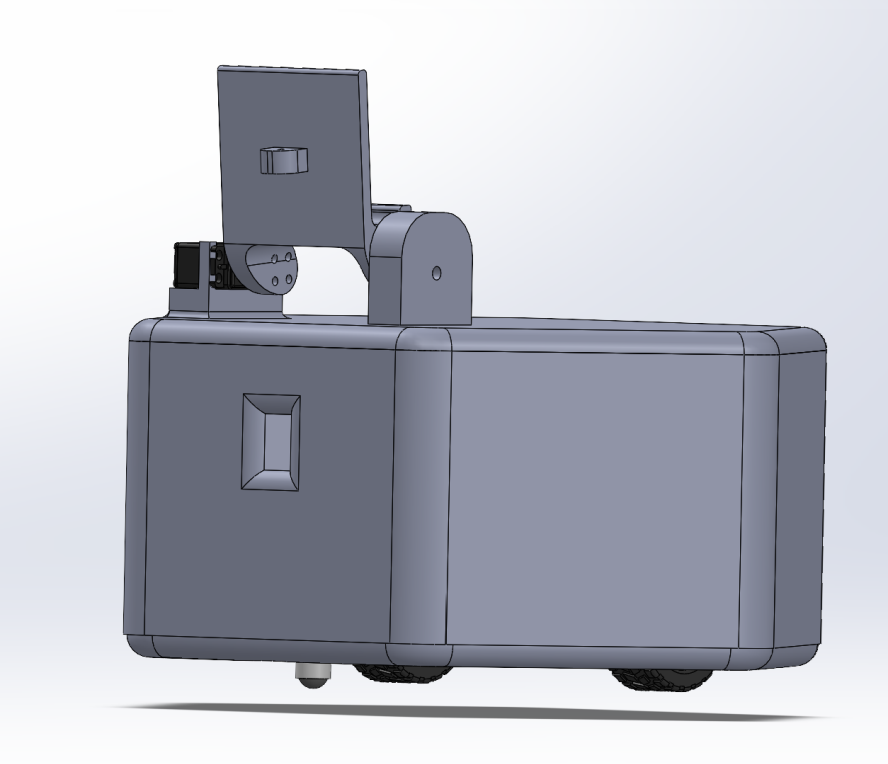


Figure 5.3: Complete 2WD mobile-base assembly concept (SolidWorks; not fabricated)

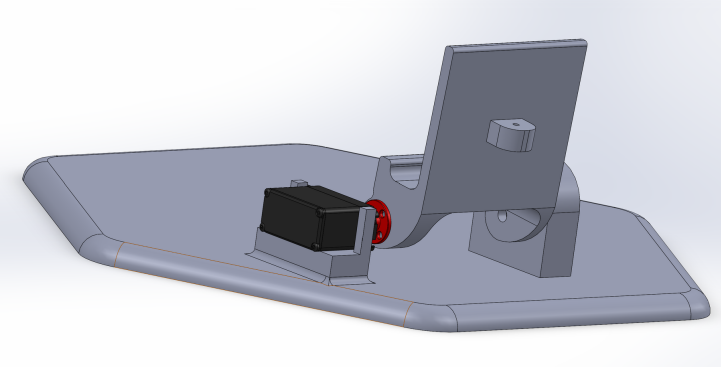


Figure 5.4: Sensing phone seated in the tilt holder

The 2WD differential drive with a castor was chosen over 4WD for cost, weight and wiring simplicity, and it is adequate for indoor hard floors. The motor driver selected is the TB6612FNG, fitted to the downsized 2WD currents, in place of the oversized BTS7960 of the July concept; shortlisted ranging sensors are the VL53L0X time-of-flight unit with three HC-SR04 ultrasonic units, and the battery basis is a 3S 1000 mAh pack. Sizing was carried out against a 3.0 kg design-basis mass: the calculated drive-torque margin is 1.27x and the tilt-servo margin is 6.9x (project sizing note).

Two honesty notes apply to everything in this section. First, the CAD is a concept complete in SolidWorks; it was not fabricated in Phase I, no tolerance drawings were produced, and no geometry was dimensionally verified. Second, the planned M2a bench rig (a static pan-tilt fixture for software bring-up) is a separate vehicle from this mobile base: its parts are listed as to order in BOM v0.1 and ordering was not confirmed at the report date.

5.3 Software Design

5.3.1 Phone Application

The phone application (Kotlin with Compose, six screens) owns all sensing. It streams ARCore visual-inertial odometry poses, Camera2 frames and IMU samples to the bridge over the versioned WebSocket protocol, and it accepts the operator's natural-language query on the second phone. The application was compiled to APK v1 in August 2026; host-side JUnit tests exist but were not re-run during report preparation.

5.3.2 Communication Bridge

The `bot_bridge` package is an asyncio WebSocket router between the phones and the PC-side consumers. It enforces a connection budget, per-client rate limiting and strict frame validation (rejecting malformed headers and oversize payloads), and it implements the 22-byte binary frame header plus JSON control-message protocol. The host-loopback end-to-end test passes (hello/ack handshake, pose plus IMU plus binary-frame round trip, session counters), and 7/7 router unit tests pass. The ROS 2 adapter is code-complete but was not executed, since no ROS 2 environment was available during Phase I.

5.3.3 Search Core

The `soma_brain` package implements probabilistic search over a 2D belief grid. The **BayesianGrid** holds an occupancy-style belief over grid cells, initialised with Gaussian priors and scored by an entropy metric. The **ConicalSensorModel** applies a field-of-view mask with detection and false-negative probabilities to each observation. The **NextBestViewPlanner** selects the heading with maximum expected information gain. The algorithm, in compact form:

1. Initialise the belief grid with Gaussian priors (uniform where no prior exists); verify the probabilities sum to one.

2. For the current pose, evaluate each candidate heading: apply the conical sensor mask, predict the posterior under detection and no-detection outcomes, and compute the expected entropy reduction.

3. Select the heading with the largest expected gain (next best view) and command the sweep.

4. Incorporate the real observation with the Bayesian update; re-check the sum-to-one invariant and record the entropy.

5. Repeat until the entropy falls below threshold or the step budget is exhausted.

Figure 5.5 shows the loop as a flowchart.

The planner is a simulation-validated Bayesian planner (unit-tested; not yet run on hardware): 12/12 unit tests pass, asserting that the planner beats a fixed sweep on entropy, that the chosen heading lies within half a field of view of the belief hotspot, that the probabilities sum to one, and that entropy decreases on a missed detection. Measured simulation behaviour is reported in Chapter 6.

5.4 Development Methodology

Development followed an iterative, review-gated process: the zeroth review, the first review and the V1 internal review each closed a design iteration. The search core was developed test-first, with the invariants above (sum-to-one, entropy monotonic on no-detection, planner beats fixed sweep) as the acceptance gates. Continuous integration runs lint, the Python unit suites and a debug build of the phone app,

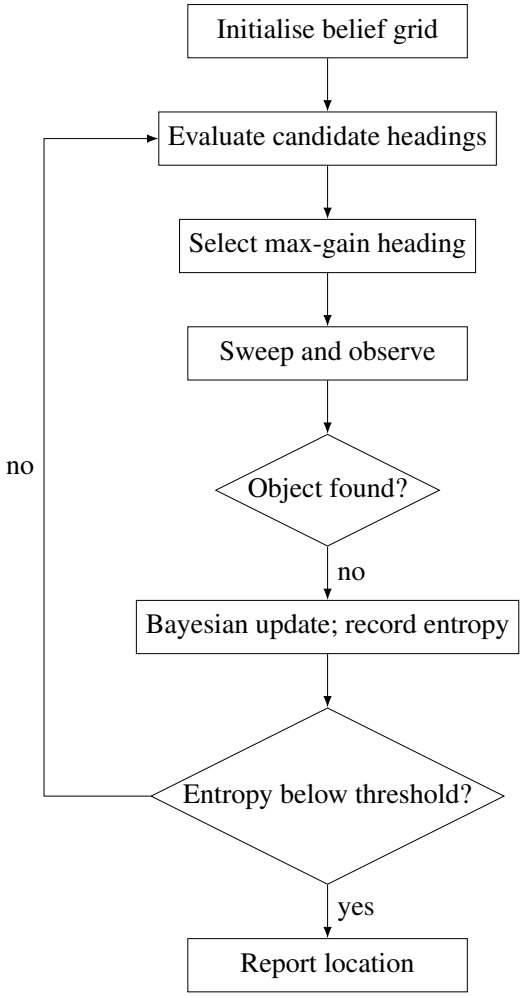


Figure 5.5: Next-best-view search loop flowchart

plus a dependency-vulnerability scan. Two repositories separate the robot software from the phone application, and design decisions are recorded as dated artefacts (approach decision, compute-placement note, micro-ROS briefing, motor/battery shortlist), a documentation-as-artefact practice that this report draws on throughout.

5.5 Phase-I Implementation Status

Table 5.1 states the verified Phase-I status of every subsystem against its evidence. Nothing in the table claims a working integrated robot.

Table 5.1: Phase-I implementation status by component

Component	Phase-I status	Evidence
Bayesian search core	Implemented, unit-tested	12/12 tests pass; simulation entropy 11.8349 to 9.3857 bits
WebSocket bridge	Host-loopback verified	7/7 tests pass; end-to-end round trip OK
ROS 2 adapter	Code-complete, unexecuted	No ROS 2 environment in Phase I
Phone app	APK v1 built Aug 2026	Release APK; JUnit suite not re-run for report
2WD base CAD	Concept complete, unbuilt	SolidWorks assembly, renders, sizing margins
URDF description	Partial	Links and display launch exist; no control or simulation
Bench rig (M2a)	Planned, parts to order	BOM v0.1; ordering unconfirmed
Perception models	Surveyed, not benchmarked	Literature fixed; no measured numbers
EKF / OctoMap / Nav2	Designed, not integrated	Phase-II items

CHAPTER 6

PRELIMINARY RESULTS

Phase I produced component-level results: a simulated search run, a verified communication bridge, an early sensor prototype, a mechanical concept with a revised cost basis, and — equally important — a precise record of what has not yet been demonstrated. All numerical claims below were re-verified at git HEAD `57eb140` (2026-10-13).

6.1 Bayesian Search Simulation

Figure 6.1 shows the regenerated simulation output: a 4×4 m world on an 80×80 grid at 0.05 m resolution, an 80-degree field-of-view sweep over 20 no-detection steps. Map entropy falls from 11.8349 to 9.3857 bits while the sum-to-one invariant holds throughout.

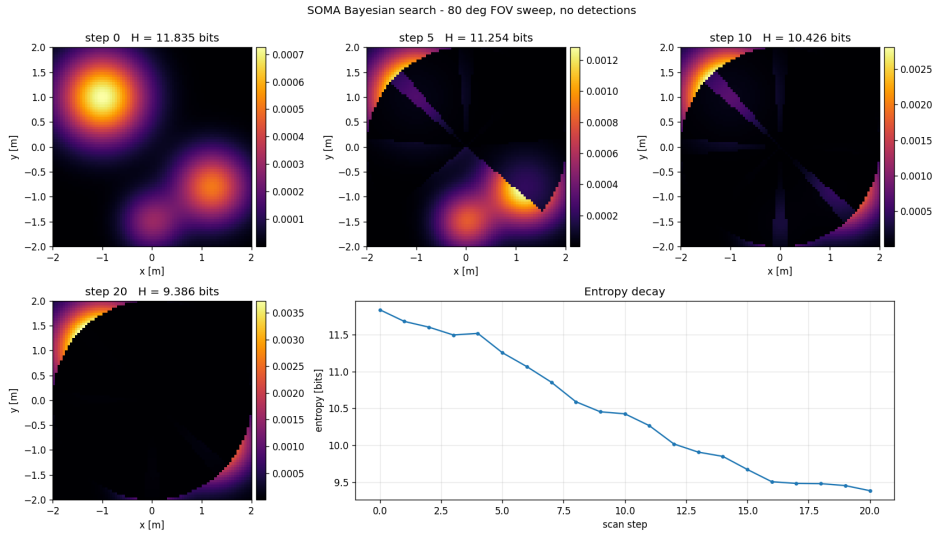


Figure 6.1: Bayesian search simulation: belief heatmaps and entropy decay over 20 steps

The accompanying unit suite (12/12 pass in `soma_brain/tests`) asserts that the next-best-view planner beats a fixed sweep on entropy reduction, that the chosen

heading lies within half a field of view of the belief hotspot, that the probabilities sum to one, and that entropy decreases on a missed detection. This demonstrates that the search logic behaves correctly in simulation. It does not prove robustness to real sensor noise, missed detections from a real detector, or odometry drift: no part of the planner has yet run on hardware.

6.2 Communication Bridge Verification

The bridge was verified by host loopback. The end-to-end test script (output: E2E OK) exercises the hello/ack handshake, a pose message, an IMU message and a binary image frame through a live WebSocket round trip, with session counters confirming one of each (hello:1, pose:1, imu:1, binary:1). The router unit suite passes 7/7, covering connection handling, rate limiting and frame validation. The ROS 2 adapter path is code-complete but untested: no ROS 2 environment was available during Phase I, so no claim is made about ROS-side behaviour.

6.3 IMU Streaming Prototype

Ahead of the full ARCore pipeline, a July 2026 prototype streamed phone IMU data to the PC over a WebSocket and recorded it to disk; four CSV sessions survive, the earliest dated 2026-07-29, with the gravity component clearly visible in the z-axis. The prototype demonstrated early that the phone-as-sensor path — the foundation of the whole hybrid architecture — was feasible with ordinary handsets and student-written code.

6.4 Mechanical Design Output

The mechanical output of Phase I is the CAD concept complete in SolidWorks (assembly, eight parts, six renders and a mesh export), not hardware. The renders in Chapter 5 are its evidence. The cost result is shown in Figure 6.2: the revised bill of materials (BOM v0.1) prices the bench rig at Rs.4,600 and the mobile base at Rs.2,790, totalling Rs.7,390 against the Rs.43,000–52,000 draft architecture — a reduction of about 83% achieved by dropping the Jetson, the 4WD transmission and the oversized drivers.

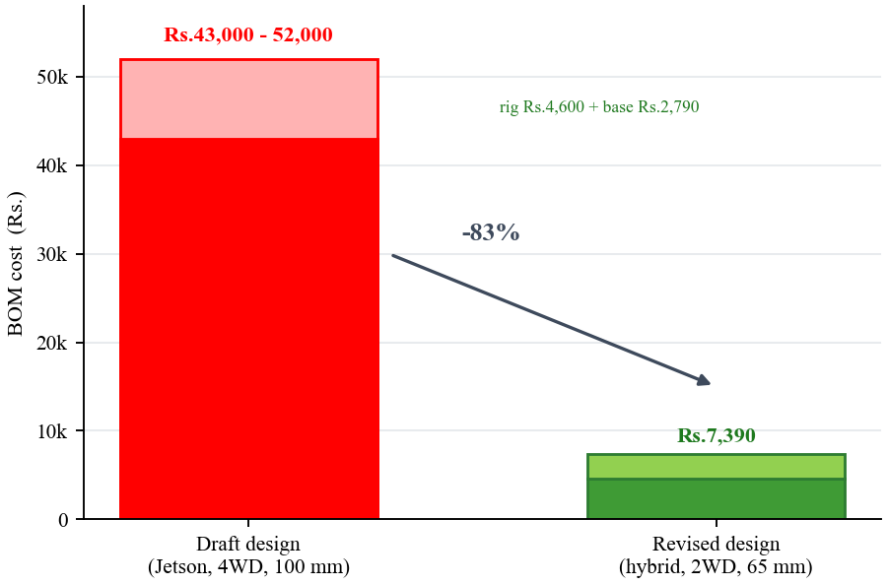


Figure 6.2: BOM cost evolution: draft architecture versus revised bench-rig and base plan prices (Rs.7,390 total)

These are plan prices from the BOM, not purchases: no parts had been ordered at the report date. What the cost result demonstrates is that the downsized 2WD hybrid architecture fits a semester budget on paper; procurement and any price movement remain Phase-II risks.

6.5 What Has Not Yet Been Demonstrated

For the record, Phase I explicitly did *not* demonstrate: ROS 2 bring-up of any node; ESP32/micro-ROS head control; EKF localisation [10]; OctoMap mapping [5] or Nav2 navigation; measured perception-benchmark numbers for YOLO-World [1], Florence-2 [13] or Grounding DINO [9]; any integrated recall demonstration on hardware; or any fabricated mechanical part. Each of these maps to a Phase-II milestone in Section 4.3.

CHAPTER 7

CONCLUSION

This report began from a concrete problem: shared laboratories lose track of their tools, and neither manual registers, RFID tagging, CCTV review nor commercial inventory robots give a small offline lab a way to ask "where is the multimeter now?" The chosen approach is the embodied semantic asset recall robot documented here: a low-cost offline robot that builds a semantic memory of a single 5×5 m room and recalls requested objects from natural-language queries, using a phone as its sensor, a local PC as its intelligence, and a Bayesian search policy to decide where to look next.

Phase I delivered the design and the components. The hybrid architecture is fixed and reviewed; the 2WD mobile-base concept is complete in CAD with sizing margins; the Bayesian search core is simulation-validated with 12/12 unit tests and a measured entropy reduction from 11.8349 to 9.3857 bits; the communication bridge passes host loopback with 7/7 unit tests; the phone application stands at APK v1; and the revised bill of materials holds the project to Rs.7,390 in plan prices, about 83% below the draft architecture.

The maturity reached is component-level, and the report has been explicit about that throughout: no integrated recall loop has run on hardware, the ROS adapter is unexecuted, the CAD is unfabricated, and no perception numbers have been measured. Integration — mechanical, electrical and software — is the Phase-II challenge.

Each limitation maps directly to a Phase-II milestone: the unbuilt base to M2b integration; the unrun ROS path and head control to the M3 live pipeline; the unmeasured detectors to the M4 benchmark of YOLO-World [1], Florence-2 [13] and Grounding DINO [9]; the untested missions to the M5 behaviour-tree implementation [2]; and operator use to the M6 dashboard. The November 2026 end-semester review is the locked date against which that plan executes. The path from this report to that demonstration is therefore defined task by task, and Phase I has supplied the evidenced foundation on which Phase II builds.

REFERENCES

[1] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 16901–16911, 2024. arXiv:2401.17270.

[2] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI: An Introduction*. CRC Press, 2018.

[3] Muhammad Fadhil Ginting, Sung-Kyun Kim, David D. Fan, Matteo Palieri, Mykel J. Kochenderfer, and Ali akbar Agha-Mohammadi. SEEK: Semantic reasoning for object goal navigation in real world inspection tasks. In *Proc. Robot. Sci. Syst. (RSS)*, 2024. arXiv:2405.09822.

[4] Qiao Gu, Alihusein Kuwajerwala, Sacha Morin, Krishna Murthy Jatavallabhula, Bipasha Sen, Aditya Agarwal, Corban Rivera, William Paul, Kirsty Ellis, Rama Chellappa, Chuang Gan, Celso Miguel de Melo, Joshua B. Tenenbaum, Antonio Torralba, Florian Shkurti, and Liam Paull. ConceptGraphs: Open-vocabulary 3D scene graphs for perception and planning. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, pages 5021–5028, 2024. arXiv:2309.16650.

[5] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. OctoMap: An efficient probabilistic 3D mapping framework based on octrees. *Autonomous Robots*, 34(3):189–206, 2013.

[6] Chenguang Huang, Oier Mees, Andy Zeng, and Wolfram Burgard. Visual language maps for robot navigation. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, London, UK, 2023. arXiv:2210.05714.

[7] Krishna Murthy Jatavallabhula, Alihusein Kuwajerwala, Qiao Gu, Mohd Omama, Tao Chen, Alaa Maalouf, Shuang Li, Ganesh Iyer, Soroush Saryazdi, Nikhil Keetha, Ayush Tewari, Joshua B. Tenenbaum, Celso Miguel de Melo, Madhava Krishna, Liam Paull, Florian Shkurti, and Antonio Torralba. ConceptFusion:

Open-set multimodal 3D mapping. In *Proc. Robot. Sci. Syst. (RSS XIX)*, 2023. arXiv:2302.07241.

[8] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. AWQ: Activation-aware weight quantization for LLM compression and acceleration. In *Proc. Mach. Learn. Syst. (MLSys)*, 2024. arXiv:2306.00978.

[9] Sicheng Liu, Renrui Zhang, Ziyu Zeng, Junjie Pei, Hao Tian, Hang Zhang, Kunchang Li, Zehuo Yu, Wenhai Wang, Chen Change Loy, Ziwei Liu, Yu Qiao, Fahad Shahbaz Khan, and Ping Luo. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. In *Proc. Eur. Conf. Comput. Vis. (ECCV)*, 2024. arXiv:2303.05499.

[10] Thomas Moore and Daniel Stouch. A generalized extended kalman filter implementation for the robot operating system. In *Proc. 13th Int. Conf. Intell. Auton. Syst. (IAS-13)*. Springer, 2014.

[11] Jeongeun Park, Taerim Yoon, Jejoon Hong, Youngjae Yu, Matthew Pan, and Sungjoon Choi. Zero-shot active visual search (ZAVIS): Intelligent object search for robotic assistants. In *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023. arXiv:2209.08803.

[12] Nur Muhammad Mahi Shafiullah, Chris Paxton, Lerrel Pinto, Soumith Chintala, and Arthur Szlam. CLIP-Fields: Weakly supervised semantic fields for robotic memory. In *Proc. Robot. Sci. Syst. (RSS XIX)*, Daegu, Korea, 2023. arXiv:2210.05663.

[13] Bin Xiao, Haiping Wu, Weijian Xu, Xiyang Dai, Houdong Hu, Yumao Lu, Michael Zeng, Ce Liu, and Lu Yuan. Florence-2: Advancing a unified representation for a variety of vision tasks. In *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pages 4818–4829, 2024. arXiv:2311.06242.

[14] Zhijie Yan, Shufei Li, Zuoxu Wang, Lixiu Wu, Han Wang, Jun Zhu, Lijiang Chen, and Jihong Liu. Dynamic open-vocabulary 3D scene graphs for long-term language-guided mobile manipulation. *IEEE Robot. Autom. Lett.*, 10(5):4252–4259, 2025. arXiv:2410.11989.

[15] Bangguo Yu, Hamidreza Kasaei, and Ming Cao. L3MVN: Leveraging large language models for visual target navigation. In *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2023. arXiv:2304.05501.